- *Static content sites*—If your site consists exclusively of static content, there is nothing to be gained by starting a Razor Pages project. You just don't need a framework with the primary purpose of generating HTML dynamically on the server.
- *Web APIs*—Razor Pages is primarily a UI-generation framework. However, Razor page handlers can return any type of content, including JSON. Nevertheless, if your application is primarily a web-based service, Razor Pages is not the correct tool to use. You should consider using MVC API controllers instead. It should be pointed out that it is perfectly possible (and easy) to mix Razor pages and API controllers in the same project, if your requirement is to generate HTML as well as to provide services over HTTP.
- *Migrating from older versions of MVC*—If you are looking to migrate an existing MVC application from previous versions of the .NET Framework to ASP.NET Core, it might make more sense to port to ASP.NET Core MVC because much of your existing code can be reused without modification. Having migrated, you can use Razor Pages for all new page-focused features within the migrated application because MVC controllers and Razor Pages happily sit together in the same application.

Razor Pages is the default project type for building page-based web applications in Visual Studio, so in all cases other than the preceding exceptions, you should use Razor Pages for page-focused applications, regardless of complexity.

ASP.NET Core has been designed with performance as a first-class feature. The framework regularly features highly, or even top, on the well-regarded TechEmpower Performance Ratings for web frameworks (https://www.techempower.com/benchmarks). So if you need a high-